

Take-Home Full-Stack Interview

Recruitment Pipeline Platform with Resume Intelligence and Analytics Dashboard

Overview

You are building a recruitment pipeline platform that allows organizations to receive, process, and manage job applications through an intelligent, data-driven interface.

The system should support two primary experiences:

- 1. Public-facing application portal**
- 2. Internal Admin dashboard with analytics and pipeline management**

The core experience is:

Admin creates a job listing with description, custom application form, and pipeline stages

- > each job posting generated is assigned a unique, public-facing URL.
- > there is no centralized portal; listings are accessible only via their direct link
- > applicant submits their application through the job-specific form
- > system extracts structured data from resume using AI (fields to extract: education, degree, work history including time periods)
- > applicant receives confirmation email
- > applicant appears in the job's dashboard
- > admin selects a job from the dashboard, reviews applicants and analytics for that role
- > admin reviews parsed profiles and original resumes
- > admin moves applicants through the job's configurable pipeline stages
- > to support an agentic future, every system action including stage transitions must be executable via API. The architecture should ensure

that the interface is entirely API-driven, allowing for seamless integration with external services and AI agents

This assignment focuses on building a polished, data-rich internal tool that would be used daily by a hiring team, not just a functional prototype.

We are evaluating:

- Dashboard design and usability
- Job management and form customization flexibility
- Template-driven role configuration
- Resume intelligence accuracy
- Pipeline workflow design
- System extensibility through API integrations
- Realistic handling of structured and unstructured applicant data

The admin dashboard is the centerpiece of this product. It should be fast, intuitive, visually informative, and genuinely enjoyable to use on a daily basis. Think of tools like Linear, Notion, or Retool in terms of interface quality and information density.

A critical requirement is that the platform must handle applicant data intelligently. Resume parsing should extract meaningful, structured information that powers filtering, grouping, and analytics throughout the dashboard.

Each job operates as its own workspace. The admin selects a job to enter its dashboard, where all applicant data, pipeline stages, and analytics are scoped to that role. This job-scoped architecture is central to the product design.

Candidates must design an experience that makes recruitment data immediately actionable.

Time Expectation

This assignment represents approximately 5 to 7 days of effort assuming effective use of AI coding tools.

Platform & Stack Requirements

Platform

Web application with a public-facing portal and an internal admin dashboard.

Recommended Stack

Layer	Recommendation
Frontend	React or any React-based framework (Next.js, Remix, etc.)
Backend	Node.js or Python
Database	Any persistent storage (Firebase, MongoDB, PostgreSQL, etc.)
Infrastructure	Google Cloud preferred, but equivalent architecture is acceptable

Suggested Integrations

Capability	Options
Resume Parsing	AI-based extraction (OpenAI, Anthropic, Google Gemini, or similar)
Email	SendGrid, Postmark, Amazon SES, Resend, or similar
File Storage	Google Cloud Storage, AWS S3, Firebase Storage, or similar

--	--

Coding Tools

AI-assisted development tools are encouraged:

- Claude Code
- Cursor
- or similar tools

Push your project to GitHub with clear setup instructions.

Before submitting:

1. Test the project in GitHub Codespaces to ensure it runs reproducibly in a clean environment.
2. Provide a hosted URL where the project is actively deployed to streamline the review and testing process.

Functional Requirements

Job Management

Admins must be able to create and manage multiple job listings. Each job is an independent workspace with its own application form, pipeline, applicants, and analytics.

Creating a Job

When creating a new job, the admin must configure:

- Job title (e.g., "Full Stack Engineer", "QA Analyst")
- Job description (provided by the admin as rich text, markdown, or via a direct link to hosted PDF or Google Doc)
- URL slug (Unique string auto-generated from title, e.g.,

????????????????????
????????????????????
????????????????????
remains fully editable by the admin)

- Job status (newly created jobs default to **Draft** so the admin can finish configuring the form and pipeline before publishing)

Job Statuses

Each job must have a status:

- **Draft** — job is being configured
- **Active** — job is published and accepting applications
- **Closed** — job is no longer accepting applications, data is preserved

Admins must be able to transition jobs between these statuses.

Application Form Builder

Each job has its own application form with customizable fields.

Default fields (always present, cannot be removed):

- First name (text, required)
- Last name (text, required)
- Email address (text, required)
- Phone number (text, required)
- Resume upload (file, required)

Custom fields:

Admins must be able to add additional fields to the application form for each job. For each custom field, the admin configures:

- Field label (e.g., "Cover Letter", "LinkedIn URL", "How did you hear about us?")
- Field type (the appropriate input type for the field — text, textarea, file upload, dropdown, etc.)
- Required or optional toggle
- For dropdown fields: the list of options
- Field order (drag-and-drop reordering)

Admins must be able to add, edit, reorder, and remove custom fields.

Examples of custom fields a job might include:

- Cover letter (file upload; support PDF only, required)

- LinkedIn URL (text, optional)
- Portfolio URL (text, optional)
- How did you hear about us? (dropdown, optional)
- Years of experience (text, optional)
- Work authorization (dropdown, required)

Custom fields can be configured directly on a job, or pulled in from a saved template (see the **Templates** section below).

Individual Job Listings & Application Interface

The platform must generate independent, standalone job pages. These pages should be configured as non-indexable (e.g., via `noindex`, `norobots`, `nofollow`, `noarchive`, `nollms`, etc tags) to ensure they are accessible only via direct link and do not appear in search engine results or a centralized directory. The pages shouldn't be scrapeable at all.

Individual Job Page

URL: (e.g.,)

Each active job has its own public page containing:

- Full job title
- Full job description (rendered from rich text or markdown)
- The job-specific application form with all default and custom fields configured by the admin

Application Submission Experience

- Form validation must provide clear, immediate feedback
- Upon successful submission, the applicant must see a confirmation message
- The applicant must receive a confirmation email
- If an applicant attempts to submit a second application for the same job using an email that has already applied to that job, the

submission must be rejected and a clear message must be shown: "An application with this email already exists for this position." (The applicant is allowed to apply to *different* jobs with the same email — the rejection is per-job, not global.)

- Visiting a closed job's URL should show a clear message that the position is no longer accepting applications

The application portal should feel professional and trustworthy. It represents the first interaction an applicant has with the organization.

Resume Parsing and Data Extraction

When a resume is uploaded, the system must extract structured data automatically.

Required Extracted Fields

The system must attempt to extract:

- Full name
- Email address
- Phone number
- Education history
 - Institution name
 - Degree or program
 - Field of study
 - Graduation year (or expected)
 - Time period
- Work experience
 - Company name
 - Job title
 - Duration
- Skills (technical and non-technical)

Parsing Requirements

- AI-powered extraction is expected (using an LLM)
- Resume parsing must run **asynchronously** after the application is submitted. The applicant must not be made to wait for parsing to complete before seeing their submission confirmation.

- While parsing is in progress, the applicant should appear in the admin dashboard with a "Parsing..." or equivalent indicator
- Once parsing completes, the extracted data must populate the applicant's profile automatically
- If parsing fails (AI service error, unparseable content, timeout, etc.), the applicant detail view must show a clear error state and provide a **retry button** that allows the admin to re-run parsing for that applicant
- The system must handle a range of resume formats and layouts
- When extraction confidence is low or fields are missing, the system should indicate this clearly rather than guessing
- The original resume PDF must always remain accessible alongside the parsed data, regardless of parsing status
- Admin users must be able to manually correct or supplement parsed data
- Add support for multiple Work experience and education history entries

Data Quality Considerations

A strong submission will demonstrate:

- Consistent extraction across different resume styles
- Graceful handling of poorly formatted resumes
- Clear distinction between parsed data and raw resume content
- Structured storage that enables filtering and analytics

Admin Dashboard and Analytics

This is the core of the product. The dashboard is the primary interface for the hiring team and should be designed for daily use.

Job Selection

When an admin logs in, they must first select a job to work with. The admin landing page should display:

- All jobs organized by status (Active, Draft, Closed)
- Key summary stats per job (total applicants, new this week, pipeline stage distribution)

- Quick access to create a new job

Once a job is selected, the admin enters that job's workspace. All subsequent views (dashboard, applicant list, pipeline, analytics) are scoped to the selected job. The admin should be able to switch between jobs easily without losing context.

Job Dashboard Overview

After selecting a job, the dashboard must present key recruitment metrics for that specific role.

Required metrics:

- Total applicants (all time and current period)
- New applicants this week and this month
- Applicants by pipeline stage (current distribution)

Required visualizations:

- Application volume over time (line or bar chart, filterable by date range)
- Distribution by institution (top schools represented)
- Distribution by degree program or field of study
- Stage-by-stage breakdown with counts

All analytics are scoped to the selected job. There is no cross-job analytics view required.

The dashboard should surface patterns and outliers without requiring the user to dig through individual profiles.

Applicant List View

The dashboard must include a comprehensive applicant list with powerful data exploration capabilities.

Sorting - users must be able to sort by:

- Application date
- Name (alphabetical)
- Institution
- Degree program
- Current pipeline stage

Filtering - users must be able to filter by:

- Pipeline stage
- Institution
- Degree program or field of study
- Skills (multi-select from skills present in the current applicant pool — applicants matching any selected skill are shown)
- Application date range

Grouping - users must be able to group applicants by:

- Institution
- Degree program
- Pipeline stage

Search - full-text search across:

- Name
- Email
- Institution
- Skills
- Any parsed resume data

Each applicant row should display enough information to assess at a glance without clicking into the detail view.

Applicant Detail View

Clicking on an applicant must open a detailed profile view.

Required elements:

- Parsed applicant information (structured display)
- Original resume viewer (embedded PDF or rendered view)
- All custom field values the applicant submitted (in the order configured by the admin)
- Current pipeline stage with ability to change it
- Activity timeline (when they applied, stage changes with timestamps)
- Admin notes section (add, edit, delete notes with timestamps)
-

Dashboard UX Standards

The admin dashboard will be judged on:

- Information density without clutter
- Speed and responsiveness of interactions
- Intuitive navigation between views (overview, list, detail, pipeline)
- Consistent visual language
- Keyboard shortcuts for common actions (bonus)
- Responsive design (should work on mobile, tablet as well as desktop)

A strong submission makes a hiring manager want to keep this open all day.

Configurable Hiring Pipeline

Each job must have its own independent, configurable pipeline. A Software Engineer job might have 5 stages while a Product Designer job might have only 4.

Pipeline Configuration

- Admins must be able to create new pipeline stages for each job
- Admins must be able to rename existing stages
- Admins must be able to reorder stages via drag-and-drop
- Admins must be able to delete stages (with confirmation and handling of applicants currently in that stage)
- When creating a new job, the system should provide sensible default stages as a starting point:
 - Applied
 - Screening
 - Assessment
 - Interview
 - Offer
 - Hired
 - Rejected
- Admins can then customize these defaults for each job before or after publishing

Pipeline Board View

The pipeline must include a Kanban-style board view.

Requirements:

- Each stage is a column
- Applicant cards are displayed within their current stage
- Applicants can be moved between stages via drag-and-drop
- Each card should show: name, institution, program, time in current stage
- Time in current stage must be displayed in compact relative form: ??, ???, ??, ??, ???, ??? (the unit auto-selects based on the magnitude)
- Moving a candidate must update their profile and log the transition in their activity timeline

The board and list views must stay synchronized. Changes in one view must be immediately reflected in the other.

Stage Transition Logging

Every stage change must be recorded with:

- Previous stage
- New stage
- Timestamp
- Admin user who made the change

This data feeds into the activity timeline on the applicant detail view.

Templates

The platform must support reusable templates that bundle together everything role-specific about a hiring workflow: the custom application form fields and the assessment questions. A single template can be applied to a job to set up both at once, saving admins from rebuilding similar roles from scratch.

Templates are global and reusable. The same template can be applied to multiple jobs, and admins can manage a library of templates over time.

Template Structure

Each template must include:

- Template name (e.g., "Software Engineer", "QA Analyst", "Product Designer")
- Optional description (admin-facing notes about what the template is for)

A template contains two parts:

1. Custom application form fields

The set of custom fields to add to the job's application form, on top of the four default fields (name, email, phone, resume) which are always present and never come from the template.

For each custom field, the template defines:

- Field label
- Field type (text, textarea, file upload, dropdown, etc.)
- Required or optional
- For dropdown fields: the list of options
- Field order

2. Assessment questions

When an admin move a candidate to a certain stage in the hiring pipeline (e.g., "Video Assessment"), they often need to send that candidate a set of interview questions — for example, to an external video assessment platform that will present those questions and collect recorded answers. Rather than re-typing these questions every time, the template stores them as a reusable list.

The full mechanics of how these questions are sent to external services are covered in the External Service Integration section below. For now, understand that this part of the template defines what questions to ask, and the integration configuration (covered later) defines when and where to send them. Each question includes:

- Question text
- Optional maximum recording duration in seconds (e.g., 120 seconds)
- Maximum number of recording attempts (default: 1, configurable per question)

When creating a new template, the assessment questions section must start with **4 questions by default**. The admin can add or remove questions freely, with the only constraint that a template must have at least one assessment question to be usable.

Either part of the template (form fields or assessment questions) can be empty if a role doesn't need it. A template with zero custom fields means the job will only have the four default fields; a template with zero assessment questions means stage-based integrations won't carry assessment data. This is fine for roles that don't require a video assessment step.

Management Requirements

- Create, edit, and delete templates
- Reorder both custom fields and assessment questions within a template via drag-and-drop
- Duplicate a template to create variations
- Preview a template from the admin interface (showing both the form fields as they would appear to applicants and the list of assessment questions)

Applying a Template to a Job

When creating or editing a job, the admin can apply a template. Doing so populates the job's custom form fields and makes the template's assessment questions available to the job's stage-based integrations.

After a template is applied, the admin can still tweak the job's custom fields independently — applying the template sets the starting point, not a permanent link. (How edits propagate is up to the candidate's discretion: linked vs. snapshot approaches are both acceptable, but the choice should be documented in the architecture overview.)

The job's integration configuration (covered in the next section) determines which pipeline stage actually fires the assessment questions out to the downstream service.

External Service Integration

The platform must support triggering external API calls when applicants move through pipeline stages.

This enables integration with external tools such as video assessment platforms, background check services, or notification systems.

Configuration

Integrations are configured per job. Each job's pipeline can have its own set of integrations.

Admins must be able to configure:

- Which pipeline stage triggers an API call
- The external endpoint URL — a freeform input field where the admin pastes any HTTPS URL pointing at the downstream service they want to integrate with
- An API key — a freeform input field where the admin pastes the bearer token issued by the downstream service (sent in the `Authorization: Bearer [API Key]` header as `Authorization: Bearer [API Key]`)
- Whether to include the job's assessment questions in the outbound payload (on/off toggle)
- Whether the integration is active or paused

Both the URL and the API key are admin-supplied at configuration time. Your platform must not assume any specific URL or hardcode a destination — it simply sends the configured request when a stage transition occurs.

Multiple stages can have different integrations configured. A single stage can have one integration.

When the "include assessment questions" toggle is on, the assessment questions from the job's applied template are embedded in the outbound API payload. This allows downstream services to receive

everything they need to run the assessment without maintaining their own copy of the questions. If the toggle is off (or the job has no template applied), the assessment block is omitted from the payload — useful for non-assessment downstream services like background checks.

API Call Specification

When a candidate enters a configured stage, the system must send an HTTP POST request to the configured endpoint.

The request must include:

Headers:



Payload:

A 10x20 grid of question marks, with the bottom-right corner missing, forming a triangular shape. The grid is composed of 10 rows and 20 columns. The bottom row (row 10) contains 4 question marks. The row above it (row 9) contains 5 question marks. The row above that (row 8) contains 6 question marks. The row above that (row 7) contains 7 question marks. The row above that (row 6) contains 8 question marks. The row above that (row 5) contains 9 question marks. The row above that (row 4) contains 10 question marks. The row above that (row 3) contains 11 question marks. The row above that (row 2) contains 12 question marks. The top row (row 1) contains 13 question marks.

[illegible]

- View all configured integrations
- See recent API call history (success/failure, timestamps, response codes)
- Test an integration with a sample payload

Testing the Integration

Since this feature triggers an outbound API call to an external service, you will need a receiving endpoint to verify it works during development and in your demo. **You do not need to build a custom receiver yourself.** Use any of the following free, public tools:

- **webhook.site** — provides a unique URL that captures and displays all incoming requests in your browser. This is the easiest option and is recommended for the recorded demo.
- **RequestBin**, **Pipedream**, or **Beeceptor** — similar webhook inspection services with free tiers
- A simple local Express, Flask, or FastAPI endpoint that logs incoming requests to the console

For your recorded demo, configure an integration with one of these tools as the destination URL, move a candidate into the configured stage, and show the request being received with the correct headers, payload, and embedded assessment data (if a template is attached).

The goal is to prove that your platform sends a correctly-formed HTTP request to whatever URL the admin configures. The downstream service does not need to do anything with the data — capturing and displaying it is enough.

Applicant Notifications

The system must send email notifications to applicants.

Required Emails

Application Confirmation

When an applicant submits their application:

- Send a confirmation email to the applicant's email address
- Include: applicant name, job title they applied for, confirmation that their application was received
- The email should be professional and well-formatted (a short, clear acknowledgement is sufficient — no need to summarize the submitted form data)

Email Requirements

- Use a real email provider (SendGrid, Postmark, SES, Resend, or similar)
- Emails must actually send (not just log to console)
- Email delivery failures should be logged

Authentication and Authorization

The platform must include proper authentication for the admin interface.
Requirements

- Admin users must log in using Google OAuth
- The application portal (career page) must be publicly accessible without authentication
- Admin sessions must be secure (httpOnly cookies or secure token storage)
- Session expiration and renewal must be handled
- At minimum, a single admin role with full access
- The system should support creating and managing admin users
- The **first admin** must be bootstrapped via an environment variable (e.g., `?? ?? ?? ?? ?? ?? ?? ?? ?? ?? ?? ?? ?? ?? ?? ??`). When that email logs in via Google OAuth for the first time, they are automatically granted admin access. This solves the bootstrap problem of needing an existing admin to invite the first one.

Optional

- Role-based access (e.g., Admin with full access, Reviewer with read-only access)
- Invite system for adding new admin users

Error Handling

The system must gracefully handle:

- Resume upload failures (file too large, unsupported format, network interruption)
- Resume parsing failures (AI service unavailable, unparseable content) — must surface a retry option as described in the parsing section
- Email delivery failures
- External API call failures (as described in the integration section)
- Authentication session expiration
- Invalid or malformed application submissions

The system should ensure:

- Errors are surfaced clearly to the appropriate user (admin or applicant)
- Partial failures do not corrupt data
- Failed operations can be retried where applicable

UX Expectations

Application Portal

- Clean, professional design
- Mobile-responsive (applicants may apply from their phone)
- Clear progress indication during submission
- Accessible form design

Admin Dashboard

This is where the product must shine.

The dashboard should demonstrate:

- Thoughtful layout and information hierarchy
- Smooth transitions and interactions

- Loading states that maintain layout stability
- Empty states that guide the user
- Consistent spacing, typography, and color usage
- Data-dense views that remain scannable
- Quick access to common actions

The admin interface should feel like a modern productivity tool, not a form-over-function prototype.

QA Expectations

Candidates must explicitly consider the following scenarios.

Testing should include:

- Creating multiple jobs with different pipeline configurations and custom form fields
- Publishing, closing, and managing job statuses
- Submitting applications with varying resume formats (single page, multi-page, different layouts)
- Verifying parsed data accuracy across multiple resumes
- Filtering and sorting with a non-trivial number of applicants (seed at least 20-30 test applicants across multiple jobs)
- Verifying that dashboard analytics are correctly scoped per job
- Drag-and-drop pipeline operations across multiple stages
- Creating templates that bundle custom form fields and assessment questions
- Applying a template to a new job and verifying the form and questions populate correctly
- External API integration with a real or mock endpoint, including the case where the assessment payload is included and the case where it is not
- Email delivery verification
- Dashboard performance with seeded data
- Concurrent user actions (if feasible)

A strong submission includes sample data across multiple jobs that makes the dashboard feel alive and demonstrates the analytics capabilities meaningfully.

Deliverables

GitHub Repository

Provide a repository containing:

- Clean project structure
- Clear README with:
 - Architecture overview
 - Local development setup instructions
 - Environment variable configuration
 - Database setup instructions
 - Seeding instructions for test data
- `env` file with all required variables documented
- API documentation (endpoints, request/response formats)

The project must run in GitHub Codespaces.

Deployed Application

Provide a live, deployed version of the application.

- Both the application portal and admin dashboard must be accessible
- Include the deployment URL in the README
- Ensure the deployed version matches the repository code

Recorded Demo

Submit a recorded demo (10-15 minutes) showing:

- Templates
 - Creating a template with custom form fields and assessment questions

- Showing the template preview
- Job management
 - Creating a new job by applying a template (showing the form fields and assessment questions populated from the template)
 - Demonstrating that custom form fields can also be added/edited directly on the job
 - Configuring required vs optional fields
 - Publishing the job (Draft to Active)
- The public careers experience
 - Browsing the careers listing page
 - Viewing a job description and application form
 - Submitting an application with resume upload
 - Receiving the confirmation email
- The admin dashboard (job-scoped)
 - Selecting a job from the admin landing page
 - Dashboard overview with metrics and charts for that job
 - Browsing the applicant list with filters, sorting, and grouping
 - Searching for applicants
 - Viewing an applicant detail profile with parsed resume data
 - Viewing the original resume
 - Adding admin notes
- The hiring pipeline
 - Viewing the Kanban board for a specific job
 - Moving applicants between stages via drag-and-drop
 - Configuring pipeline stages (add, rename, reorder, delete)
 - Showing that different jobs have different pipeline configurations
- External service integration
 - Configuring an integration for a pipeline stage
 - Toggling on "include assessment questions" in the integration
 - Demonstrating the API call being triggered on stage transition (showing the assessment data embedded in the outbound payload)
 - Showing the integration log (success/failure history)
- Reporting
 - Showing analytics visualizations with seeded data
 - Exporting data to CSV

Present the demo as if showing the product to a non-technical hiring manager who will use this tool daily.

System Architecture Overview

Deliver a live technical walkthrough explaining:

- Overall system architecture
- Job and multi-tenant data model design
- Template data model — how a template's form fields and assessment questions are stored, and how applying a template to a job propagates (linked vs. snapshot)
- Application form customization architecture
- Asynchronous resume parsing pipeline and AI integration
- Dashboard rendering strategy
- Pipeline state management (per-job pipelines)
- External API integration design (including how template assessment content is embedded in outbound payloads)
- Authentication flow (including the env-var bootstrap for the first admin)
- Email delivery system
- File storage strategy

Also explain:

- How the system would handle scaling to thousands of applicants
- How resume parsing accuracy could be improved over time
- Key architectural tradeoffs made during development
- What you would change or add given more time

Evaluation Criteria

Product Understanding

We evaluate:

- Job management and form customization flexibility
- Application portal professionalism and usability
- Job-scoped dashboard information architecture

- Analytics depth and visual clarity
- Pipeline workflow intuitiveness
- Practical UX decisions that serve a daily user

Engineering Judgment

We evaluate:

- Data model design for multi-job, structured applicant data
- Application form builder implementation
- Resume parsing approach and accuracy
- Clean API design
- External integration reliability patterns
- Authentication implementation
- Database query efficiency (especially for filtering, sorting, grouping across job-scoped data)
- Code organization and maintainability

Dashboard and Analytics Quality

We specifically evaluate:

- Visual polish and consistency
- Information density vs. clarity balance
- Interactivity and responsiveness of data exploration
- Chart and visualization effectiveness
- Filter and sort implementation quality
- How well the dashboard tells the story of the recruitment data

End-to-End Experience Quality

We care about:

- Complete, working flow from application to pipeline management
- Reliability across the entire workflow
- Clear communication of system state
- Useful, actionable outputs
- No broken or dead-end flows
- Production-level deployment

This exercise evaluates your ability to build a data-rich internal tool that manages multiple job listings, transforms unstructured input (resumes and applications) into structured, actionable information, and presents it through a polished and powerful admin interface.

The best submissions will feel like a product a hiring team would genuinely prefer to use every day.